Operating Systems

ASSIGNMENT 1

CO1: *Apply the concepts of operating system types, structures and system calls to demonstrate their functions in various computing environments – history of operating systems, mainframe, desktop, multiprocessor, distributed, clustered, real-time and handheld systems, OS structure, services, system calls, system design, CPU states, memory hierarchy and I/O systems.*

Assignment Title: Evaluate and Justify the Operating-System Design of a Smart Hospital (Total Marks - 50)

Nature of Assignment: Individual **Duration:** 1 Week
Submission Date: 02/11/2026

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

- **Every student must individually execute every task in this assignment**, including all "Complete the Code" programs.
- Create a GitHub repository named **https://github.com/192525294simats/CSA0421-Operating-system.git-Assignment1** and share it with the course faculty (public, or private with the faculty added as a collaborator).
- Keep one folder for each task (for example: Task01, Task02 … or Q1, Q2 …), containing the C source files, shell scripts, configuration files (Dockerfile, YAML), command outputs and screenshots.
- Include a README.md with your name, register number, compiler and version (for example gcc 13), the Linux distribution and version used, the exact gcc command used to compile (for example gcc -std=c11 -Wall -Wextra -pthread), the steps to run each program, and a short note of the result of each task.
- Commit your work progressively – at least one commit for each task. The commit history will be checked; a single upload of all files on the last day will not be accepted.
- Submit the GitHub link below and on the cover page of your report. **An assignment submitted without a working GitHub link will not be evaluated.**

GitHub Repository Link: https://github.com/DHARA2626/slot-A-OS.git

How to Answer the Additional Questions

Complete the Code: Copy the given code into your repository, replace every TODO with working code without changing the rest of the program, execute it and compare your output (or result) with the expected output.

Alternative Concept and Complexity Analysis: Describe how the solution would change if the alternative concept were used, with code or pseudocode, and analyse and compare the time complexity and space complexity of both solutions using Big-O notation, supported by your own measurements wherever possible.

Scenario — Operating Systems for a Smart Hospital

A 500-bed hospital is replacing its computer systems. It needs: bedside patient monitors that must raise an alarm within 10 milliseconds of an abnormal reading; nurses' handheld tablets; doctors' desktop workstations; a central database server for patient records that must never stop; an imaging server that processes MRI scans using many processors; and a link to two partner hospitals that share records.

The IT team has proposed using one general-purpose operating system everywhere. The clinical staff are concerned that alarms may be delayed, the database may stop during updates, and the tablets' batteries will not last a shift.

The number of devices may grow:

1 ward (40 monitors, 10 tablets) → 1 hospital (500 monitors, 120 tablets, 80 workstations) → a group of 3 hospitals sharing records

You are required to evaluate the requirements of every part of the system, judge which type and structure of operating system suits each, examine how programs obtain OS services through system calls, and defend a complete operating-system plan for the hospital.

Design Challenge:

The hospital is adding remote patient monitoring at home and a cloud-based records service.

Re-evaluate your original design and determine whether it should be retained, modified or redesigned.

TASK 1 – Evaluate the Requirements of Each Computing Environment 4 Marks

Bloom's Level: K5 – Evaluate

Evaluate what each part of the hospital needs from its operating system.

1. Ranked requirements (1 = most important)

Environment	Response time	Reliability	Throughput	Battery	Security
Bedside monitor	**1**	2	5	4	3
Nurse tablet	3	4	5	**1**	2
Doctor workstation	**1**	3	4	5	2
Database server	4	**1**	3	5	2
Imaging server	4	2	**1**	5	3
Partner link	4	2	3	5	**1**

Justification of each ranking

Environment	Why this order
Monitor	The 10 ms alarm deadline is the whole purpose of the device, so response time is first. A monitor that crashes gives no alarm at all, so reliability is second. Readings are patient data, so security comes third. It runs on mains power with a backup battery,

Environment	Why this order
	and its data rate is tiny (a few hundred bytes per second), so battery and throughput come last.
Tablet	Nurses carry it for a whole shift, so a dead battery means no bedside medication checks. Battery is first. Tablets are easily lost and hold patient data, so security is second. Touch response of about 100 ms is enough, and a crash is annoying but recoverable.
Workstation	Doctors work interactively and wait on every click, so responsiveness is first. It is mains-powered, so battery is irrelevant. Security is second because it holds open patient sessions.
Database server	"Must never stop" puts reliability first. It holds every patient record, so security (access control, integrity) is second. Throughput is third for many simultaneous queries. A few extra milliseconds per query are acceptable.
Imaging server	An MRI scan produces hundreds of MB, so processing many scans on many processors puts throughput first. Reliability is second, because a failed run costs a repeat scan. Nobody waits on a millisecond-level deadline.
Partner link	Records cross an untrusted network, so security (authentication, encryption, auditing) is first. Reliability is second, because it must retry and not lose or duplicate records.

2. Conflicting requirements

Conflict	Where it appears	Why
Fast response vs many users	Workstations, database server	Time-slicing many users adds context switches and queueing. Response time grows as users are added.
Fast response vs throughput	Imaging server sharing a machine with anything urgent	Long CPU bursts (high throughput) block short urgent tasks. Large time quanta help throughput and hurt response.
Battery vs response	Tablets, home devices	Deep sleep saves power but wakes slowly. Polling or a fast clock responds quickly but drains the battery.
Reliability vs updating	Database server	Patches usually need a restart, but the server must never stop. This is what the clinical staff fear.
Security vs speed of access	Tablets, workstations	Encryption, auto-lock and checks cost time and battery. In an emergency, a nurse needs fast access.
Security vs openness	Partner link	Sharing records needs open connections, and every open connection is an attack surface.

3. Requirements critical to patient safety

1. **The monitor's 10 ms alarm deadline.** A late alarm is a direct clinical hazard. This is a hard real-time requirement.
2. **Database availability and integrity.** An unreachable or corrupted record can mean wrong drugs or missed allergies.

3. **Record isolation.** One patient's data must never be shown or written to another patient's chart.
4. **Tablet battery lasting a full shift.** Otherwise bedside checks fail mid-shift.

Workstation and imaging throughput matter for efficiency, but a delay there is not a direct danger.

4. Lessons from OS history

Era	Lesson	Where it applies
Batch	Maximise CPU use by queueing jobs with no user interaction. It has no deadlines and no responsiveness.	Imaging server: queue MRI jobs and keep every processor busy. It must not be used for alarms.
Time-sharing	Share the CPU fairly between many users, with protection between them. Response degrades as users are added.	Database server and workstations: many users, strong isolation. It cannot promise a deadline.
Personal	Dedicate the machine to one user and optimise for a responsive GUI. Early versions had weak protection.	Doctor workstations. Security has to be added back (accounts, memory protection).
Mobile	Power management, sandboxed apps and wireless are first-class concerns.	Nurse tablets and the later home devices.
Real-time/embedded (the fifth lesson)	Predictability matters more than average speed.	Monitors.

Key point: no single era's design gives all six environments what they need.

5. Alternative: one general-purpose OS on every device

Using one general-purpose OS (a standard desktop/server Linux or Windows) everywhere would fail the first requirement and cost more everywhere else. The figures below are typical orders of magnitude, not measurements from this assignment.

Criterion	Single general-purpose OS	OS matched to each environment
Worst-case alarm time	Unbounded in theory. Delay = interrupt latency + scheduling delay + processing, and the scheduling term depends on load: fairness-based scheduling, page faults, swapping, long non-preemptible kernel sections, background updates. Under heavy load, tens to hundreds of ms is plausible, which breaks the 10 ms deadline.	Bounded. A hard RTOS (FreeRTOS, QNX) guarantees worst-case latency in microseconds to under a millisecond, so the deadline is provable (10 ms ≫ worst case).
Memory per device	Hundreds of MB to several GB (kernel, services, GUI). That is wasteful on 500 monitors, and extra memory also means extra power and cost.	RTOS kernels need kilobytes. A monitor needs only its sensor, alarm and display code.
Administration effort	One image and one skill set to maintain, which is attractive. But every device carries services it never uses, which means more to patch and more to attack. One flaw affects all 500+ monitors at once, and one patch window must suit both the "never stop" server and the monitors.	More skills and several images, but each is small and only what is needed is present. Effort scales with the number of OS classes (about 3 to 4), not the number of devices, if managed centrally.

Conclusion: Reject the IT team's single-OS proposal. Use a real-time OS for monitors, a power-managed mobile OS for tablets, and general-purpose OSes for workstations and servers, with common central management. Tasks 2 to 4 develop which type and structure fits each.

TASK 2 – Judge the Type of Operating System for Each Environment 3 Marks

Bloom's Level: K5 – Evaluate

Compare the types of operating systems for the hospital.

1. Comparison of OS types

Type	Main idea	Strength	Weakness	Role in the hospital
Mainframe	Large, very reliable machine for batch and many-user work	Very high reliability and I/O capacity	Very expensive and needs specialist staff	Rejected as overkill for a 500-bed hospital
Desktop	One user, responsive GUI	Familiar, good application support	Weak real-time behaviour, a lot of unneeded services	Doctor workstations
Multiprocessor	Several CPUs share memory under one OS	More throughput on one machine	The CPUs compete for memory and locks	Imaging server
Distributed	Machines on a network act as one system	Resource sharing, can span sites	Network delay and failures, harder security	Partner-hospital link
Clustered	Several machines share storage and monitor each other	Fail-over, high availability	Extra cost and complexity	Database server
Hard real-time	A missed deadline is a failure	Guaranteed worst-case response	Small feature set, low average throughput	Bedside monitors
Soft real-time	A missed deadline degrades quality	Flexible and richer	No guarantee	Central display dashboards (not alarms)
Handheld	Small device, power-limited, touch UI	Power management, app sandboxing	Limited memory and CPU	Nurse tablets

2. Best fit for each environment, with rejected alternatives

Environment	Chosen type	Why	Rejected alternatives and reasons
Bedside monitor	Hard real-time (embedded RTOS)	The 10 ms alarm deadline must be guaranteed, and a missed one is a safety failure	**Soft real-time:** it gives no guarantee. **Desktop:** unbounded delay and too much memory.
Nurse tablet	Handheld (mobile OS)	Aggressive power management for a full shift, touch interface, sandboxed apps	**Desktop:** services drain the battery. **Real-time:** no rich UI needed.
Doctor workstation	Desktop (general-purpose, multi-tasking)	Responsive GUI and application support for records and viewers	**Mainframe terminals:** poor user experience and high cost. **RTOS:** wrong feature set.
Database server	Clustered (multiprocessor nodes with fail-over)	"Must never stop": if one node fails, another takes over within seconds	**Single large machine:** single point of failure. **Mainframe:** high cost.

Environment	Chosen type	Why	Rejected alternatives and reasons
Imaging server	Multiprocessor (SMP)	MRI processing splits well across many processors with shared image data	**Single CPU:** too slow. **Cluster:** needs image data copied between nodes.
Partner link	Distributed	Records are shared between three sites over a network	**Isolated systems:** no record sharing. **Shared single server:** one site's outage affects the others.

3. Symmetric vs asymmetric multiprocessing (imaging server)

	Symmetric (SMP)	Asymmetric (AMP)
Design	Every CPU runs the OS and applications and shares memory	One master CPU runs the OS, and the others are given work
Load balance	Automatic, any CPU takes any ready job	Master can become the bottleneck
Reliability	If a CPU fails the others continue	If the master fails, the system stops
Complexity	More: locking on shared kernel data	Simpler
Verdict	**Chosen.** MRI slices are independent jobs, so SMP keeps all processors busy and tolerates a CPU failure	Rejected because the master limits scaling as processors increase

4. Clustered database server with fail-over

- Two or more nodes share storage, or replicate it.
- The nodes send each other heartbeat messages.
- If the active node stops responding, a standby takes over its IP address and services, usually within seconds.
- Updates can be rolling: patch the standby, switch over, then patch the other node. This answers the clinical staff's worry that the database stops during updates.
- A caution: the shared storage or network becomes the new single point of failure unless it is also duplicated.

5. Alternative: one large multiprocessor machine instead of a cluster

The figures below are illustrative.

Criterion	Single large multiprocessor	Cluster with fail-over
Availability	One failure (power supply, board, OS crash) stops everything, which is a single point of failure. If one machine is 99.9% available, that is about 8.8 hours of downtime a year.	If two nodes fail independently, both being down at once has probability about 0.001 × 0.001, or roughly 99.9999% in the ideal case. In practice each fail-over adds a few seconds of interruption, and shared storage must be duplicated.
Cost	Large SMP machines are priced super-linearly (double the processors costs far more than double). The hardware is costly to replace.	Commodity nodes cost less per unit. There is extra cost for clustering software, networking and duplicate storage.
Throughput (time)	Limited by Amdahl's law: speedup = 1 / ((1 − p) + p / N). If 95% of the work is parallel, 8 processors give about 5.9×, 16 give about 9.1×	Throughput grows by adding nodes, but only if the data is split or replicated well. Cross-node queries cost network time.

Criterion	Single large multiprocessor	Cluster with fail-over
	and 64 give about 15.4×. Lock contention on shared memory makes it worse.	
Memory (space)	One large shared memory, ideal for big working sets and caching.	Each node has its own memory. Data is partitioned or replicated, so total memory is larger but some of it is duplicated.

Conclusion: the cluster wins on the requirement that matters most for this server, which is never stopping. The single machine would only be better if the working set were too big for one node's memory.

TASK 3 – Evaluate Operating-System Structures 4 Marks

Bloom's Level: K5 – Evaluate

1. The five structures compared

Structure	How it is built	Reliability	Performance	Ease of updating	Examples
Simple / monolithic	Whole OS in one kernel-mode program; every part can call every other	**Low:** any bug in any part (e.g. a driver) can crash the whole system	**High:** a service request is a plain function call inside the kernel	**Poor:** change means rebuild and reboot	Early UNIX, MS-DOS
Layered	OS split into layers; each layer uses only the layer below	Medium: bugs are easier to localise	Medium to low: a request passes through several layers	Medium: a layer can be replaced if its interface stays the same	THE system, network protocol stacks
Microkernel	Kernel keeps only scheduling, IPC and basic memory management. Drivers, file system and network run as user-space servers	**High:** a crashed driver is just a process that can be restarted	Lower: services talk by message passing, which adds copies and context switches	**Good:** restart one server without a reboot	QNX, MINIX 3, L4
Modular (loadable kernel modules)	Monolithic core, but drivers and file systems load and unload at run time	Medium: modules still run in kernel mode, so a faulty module can still crash the kernel	**High:** same as monolithic once loaded	**Good:** add or replace a module without rebuilding	Linux
Hybrid	Mix: microkernel-style design, but many services kept in the kernel for speed	Medium	Medium to high	Medium	Windows NT family, macOS

2. Which structure suits which environment

Environment	Chosen structure	Why
Bedside monitor	**Microkernel real-time OS** (QNX-style; FreeRTOS for the very smallest sensor nodes)	A driver fault must not stop the alarm path. Services can be restarted individually. The kernel is small and its worst-case behaviour is easy to analyse for the 10 ms deadline.
Database server	**Modular monolithic kernel** (Linux with loadable modules)	Heavy disk and network I/O needs the lowest per-request cost. Wide driver and storage support. Mature database software.

Environment	Chosen structure	Why
		Modules and kernel live patching reduce restarts, and the cluster covers the rest (rolling updates).

Why not the other way round? A pure monolithic kernel on the monitor loses fault containment. A microkernel on the database server adds message-passing cost to every I/O request and limits software availability.

3. How real operating systems are structured

OS	Structure
Linux	Monolithic kernel with loadable modules. Scheduler, memory manager, file systems and most drivers run in kernel mode in one address space.
Windows (NT family)	Hybrid. A hardware abstraction layer sits under a kernel and an “executive” of managers (memory, process, I/O). Many services and drivers run in kernel mode for speed. Subsystems (Win32) run in user mode.
Android	Layered stack: a modified Linux kernel (monolithic with modules), then a hardware abstraction layer, then native libraries and the Android Runtime, then the application framework, then apps. Each app is sandboxed in its own process.
FreeRTOS	A tiny kernel (scheduler, queues, semaphores, timers) linked together with the application into one program. By default there is one address space and no memory protection (an optional MPU port adds some).
QNX	True microkernel. Drivers, file systems and networking are separate user-space processes that talk to the kernel and each other by message passing. A supervisor can restart a failed service.

4. Alternative: monolithic kernel instead of a microkernel on the monitors

The figures below are typical orders of magnitude, not measurements from this assignment.

Criterion	Monolithic kernel	Microkernel
Time cost of a service request	One system call: a mode switch plus a function call, typically a fraction of a microsecond up to about a microsecond.	The request goes client → server → (driver) and back: at least two message exchanges and context switches, typically a few microseconds.
Does the time difference matter?	The 10 ms alarm deadline is thousands of times larger than either figure, so IPC cost does not threaten the deadline. Predictability matters more than speed here.	
Kernel memory	Typically several MB or more, because all drivers and subsystems are inside it.	Tens to a few hundred KB for the kernel itself, plus only the servers actually needed.
Effect of a faulty driver	The driver runs in kernel mode with full privilege. It can overwrite kernel memory, causing a kernel panic that stops the monitor and silences alarms. Studies of OS crashes have repeatedly found drivers to be a leading cause.	The driver is an ordinary process. If it crashes, the supervisor restarts it, other services (including alarm logic) continue, and only that device is briefly unavailable.

Conclusion: the monolithic design is a little faster per request but gains nothing that matters within a 10 ms deadline, and it exposes the alarm function to every driver bug. The microkernel is the safer choice for the monitors.

Summary for your report

Environment	Structure	Key reason
Monitors	Microkernel RTOS	Fault isolation, small, bounded latency
Database server	Modular monolithic (Linux)	I/O speed, driver and software support, live patching

TASK 4 – Evaluate the Services an Operating System Must Provide 4 Marks

Bloom's Level: K5 – Evaluate

1. Service table for every environment

● Full ◐ Reduced ○ Removed

Service	Monitor	Tablet	Workstation	Database server	Imaging server	Partner link
User interface	◐ fixed display and alarm indicators, no GUI	● touch GUI	● full GUI	◐ remote command line only	◐ remote command line, images viewed on workstations	◐ admin command line only
Program execution	◐ fixed set of tasks loaded at boot	● sandboxed apps	●	● many concurrent processes	● many parallel jobs	◐ gateway services only
I/O operations	● sensor, alarm and display only	● touch, Wi-Fi, barcode, camera	●	● high-speed disk and network	● scanner, disks (DMA)	◐ network only
File systems	◐ small log and configuration in flash	◐ encrypted app storage	●	● journaling, durable	● large-file storage	◐ queue and logs
Communication	◐ minimal protocol to the nurse station	● Wi-Fi	●	● clients and cluster heartbeat	●	● encrypted channels to partners
Error detection	● sensor fault, watchdog, self-test	◐	●	● ECC, checksums, RAID	●	●
Resource allocation	◐ fixed priorities, memory allocated at start	● with power management	●	● CPU, memory and I/O quotas	● multi-CPU scheduling	◐
Accounting	○ (alarm and event log only)	◐	◐	● audit of who read what	◐	● audit of cross-site access
Protection	◐ task isolation by the kernel	●aapp sandboxing	●	●	●	●
Security	signed firmware, authenticated configuration	●encryption, auto-lock, remote wipe	●	●login, encryption at rest	●	● mutual authentication, encrypted link

2. Services removed from the bedside monitor

Removed service	Why removing it helps
GUI and windowing system	Saves memory and removes a large amount of code that could fail

Removed service	Why removing it helps
General multi-user login and accounting	One fixed function, so there are no users to manage
General file system and shell	Only a small log and configuration area is needed
Virtual memory and swapping	Page faults and swapping make response time unpredictable. All memory is allocated at start-up.
Dynamic program loading, package manager	The software is fixed and signed, which makes it smaller, faster and a smaller attack surface
Full network stack	Keep only the minimal protocol needed to reach the nurse station

Result: a smaller and faster OS whose behaviour can be analysed against the 10 ms deadline.

3. Services essential for the database server

1. **File system with durability:** journaling and flush-to-disk guarantees, so a crash never loses a committed record.
2. **Process and thread management and scheduling:** many simultaneous queries.
3. **Memory management with a large buffer cache:** fast record look-up.
4. **I/O subsystem:** fast disk and network throughput.
5. **Error detection:** ECC memory, checksums, RAID, so a fault is caught before it corrupts a record.
6. **Protection and security:** authentication, access control, encryption at rest.
7. **Accounting and audit trail:** a record of who accessed which patient.
8. **Communication:** client connections and the cluster heartbeat for fail-over.

The GUI, sound and similar services are not needed.

4. How the OS protects one patient's records from another user's programs

1. **Authentication:** every process runs on behalf of a logged-in user (or service account).
2. **Hardware memory protection:** each process has its own address space, enforced by the CPU in user mode. A program that tries to read another process's memory is stopped with a fault.
3. **System calls as the only door:** a program can reach a record file only through open(). The kernel checks the user's permissions at that moment and returns a "permission denied" error if the user is not allowed.
4. **File permissions and access-control lists:** the record files are readable only by the database service account, not by ordinary users. Optional mandatory access control (SELinux-style) adds a second layer.
5. **Database-level control:** OS permissions work per file, so the database enforces patient-level and role-level rules (doctor of this ward, nurse on duty).
6. **Encryption and audit:** records are encrypted on disk and in transit, and every access is logged.

5. Alternative: tablets running a full desktop OS with every service enabled

The figures below are illustrative estimates, not measurements.

Criterion	Full desktop OS	Reduced handheld OS
Memory (space)	Several GB of RAM already in use at idle, from background services and a full GUI	Roughly a fraction of that, with services suspended when idle
Storage (space)	Tens of GB for the OS, services and drivers the tablet never uses	A few GB
Start-up time	Many services to start, so noticeably slower to be ready	Faster boot and fast resume from sleep
Battery life (time)	Background services keep the CPU awake and the screen and radios busy, so the battery can fall well short of a full shift	Aggressive power management (suspend, wake-lock control) is designed to last a shift

Conclusion: The full desktop OS wastes memory and storage and risks failing the battery requirement, so the reduced handheld OS is the right choice for the tablets. This also answers the clinical staff's concern about battery life.

Summary for your report

Environment	Services kept	Services removed or reduced
Monitor	I/O, error detection, minimal communication	GUI, accounts, general file system, virtual memory, dynamic loading
Tablet	Touch UI, Wi-Fi, encryption, app sandbox, power management	Unneeded desktop services
Database server	Durable file system, scheduling, protection, audit, cluster communication	

TASK 5 – Examine System Calls 4 Marks

Bloom's Level: K5 – Evaluate

1. What a system call is

A system call is the controlled entry point through which a user program asks the kernel to do something it is not allowed to do itself, such as touching a disk, a device, the network or another process. The hardware keeps user programs out of kernel mode (see Task 8), so the program must ask the kernel instead. In the hospital this applies to the database server reading a patient record, the monitor writing to its sensor driver, and the tablet sending data over Wi-Fi.

2. Categories of system calls

Category	Purpose	Linux examples	Hospital example
Process control	Create, run, end and wait for processes; allocate memory	fork, execve, wait, exit, brk/mmap	The shell on the DB server starts a backup job

Category	Purpose	Linux examples	Hospital example
File management	Create, open, read, write, close and delete files	open, read, write, close, unlink	Reading a patient record
Device management	Request or release devices; read, write and control them	ioctl, read/write on /dev/...	Reading the pulse-oximeter sensor
Information maintenance	Get or set time, date and system or process attributes	getpid, time, gettimeofday, uname	Time-stamping an alarm
Communication	Message passing or shared memory between processes or machines	socket, send, recv, pipe, shmget	Sending records to a partner hospital
Protection	Control access to resources	chmod, chown, umask, setuid	Restricting a record to its doctor

3. What happens when a program calls read()

Take the call read(fd, buf, 64).

1. **Library call.** The program calls the C library wrapper read(). This is an ordinary function call, still in user mode.
2. **Set-up.** The wrapper puts the system-call number into a register (rax on x86-64, number 0 for read). It puts the arguments into registers (rdi=fd, rsi=buf, rdx=count).
3. **Trap.** The wrapper executes a special instruction (syscall on x86-64, int 0x80 on old x86, svc on ARM). This is a software interrupt, or trap.
4. **Mode switch.** The hardware sets the mode bit to kernel mode, saves the user's program counter and flags, and jumps to the kernel's system-call entry point. The kernel switches to its own stack.
5. **Kernel work.** The kernel checks that the call number is valid and then looks it up in the system-call table. It then calls sys_read, which:
    - validates the arguments (fd is open, buf lies in the user's memory, permissions allow reading);
    - finds the data in the page cache, or asks the disk driver for it, which may block the process while another runs;
    - copies the data into the user's buffer.
6. **Return.** The kernel puts the result (bytes read, or -errno) in rax, restores the user state, and executes sysret, which sets the mode bit back to user mode.
7. **Back in user code.** The wrapper returns the value to the program. A negative result is turned into -1 with errno set.

User mode : program → read() wrapper → [syscall] ← return value → program

| trap, mode bit = 0 ▲ sysret, mode bit = 1

Kernel mode: entry → table[0] → sys_read → copy data to user buf

4. How parameters are passed to the kernel

Method	How it works	Used by
Registers	Arguments go straight into CPU registers	Linux x86-64 (up to 6 arguments); the fastest method
Block (table) in memory	Arguments are stored in a memory block, and its address is passed in a register	Used when there are more arguments than registers, e.g. some Solaris calls; older Linux for some calls
Stack	The program pushes the arguments and the kernel pops them	Some other OSes and older ABIs

Linux uses registers for speed, and falls back to a memory block when a call has more parameters than registers. The block and stack methods allow any number of parameters.

5. Evidence: strace on a real program

The program (rd.c) opens a file, reads it once, writes the data to the screen and closes it:

c

int fd = open("/etc/hostname", O_RDONLY);

ssize_t n = read(fd, buf, 64);

write(1, buf, n);

close(fd);

Output of strace ./rd (last lines, run on Linux):

openat(AT_FDCWD, "/etc/hostname", O_RDONLY) = 3

read(3, "vm\n", 64) = 3

write(1, "vm\n", 3vm

) = 3

close(3) = 0

exit_group(0) = ?

+++ exited with 0 +++

How to read it:

- openat(...) = 3: the C open() becomes the openat system call, and the kernel returns file descriptor 3. (0, 1 and 2 are stdin, stdout and stderr.)
- read(3, "vm\n", 64) = 3: this is the read() above. The arguments are fd 3 and a 64-byte buffer. The return value 3 means 3 bytes were read.
- write(1, "vm\n", 3) = 3: 3 bytes were written to fd 1 (the screen). The word vm printed in the middle of the line is the program's own output mixing with strace's.
- exit_group(0): the process-control call that ends the program.

strace -c gives a summary table of the number of calls, time and errors per system call. This is used in Task 6 and Task 10.

6. Alternative Concept and Complexity Analysis: fopen/fread/fwrite instead of open/read/write

Solution. The C library functions fopen, fread and fwrite keep a buffer in user space (the FILE structure). fread copies from this buffer and only calls the read() system call when the buffer is empty. fwrite fills the buffer and only calls write() when it is full or on flush or close.

Number of system calls for a file of n bytes

- Direct system calls with a buffer of b bytes: about **n / b** read() calls, plus one extra that returns 0 at end of file.
- Library buffering, with a typical buffer of 4096 bytes: about **n / 4096** read() calls, however small the program's own reads are. Even byte-by-byte fgetc costs only n / 4096 system calls.

Measured with n = 10,000 bytes (strace -c -e trace=read):

Method	Buffer	read() calls (strace)	Theory (n/b, rounded up, +1 for EOF)
read() directly	1 byte	10,002	10,001
read() directly	512 bytes	22	21
read() directly	4096 bytes	5	4
fgetc loop (library buffer)	4096 (internal)	5	4

strace shows one more read than the program's own count in each case. The extra call is the dynamic loader reading the ELF header of the shared library, which is not the program's own read of the data. The pattern matches the theory exactly.

The result: a byte-at-a-time program makes **10,002 calls with raw read()** but only **5 with fgetc**, about 2,000 times fewer. Each saved call avoids a trap, a mode switch and a return, so the library version is much faster for small reads.

Space. The library allocates a buffer per open file, usually 4 KB (BUFSIZ is 8 KB on glibc, and strace shows 4096-byte reads for a regular file). For 1,000 open files that is about 4 MB extra. This is a small cost, which is why library buffering is nearly always worth it.

Evaluation.

Criterion	Direct read/write	fread/fwrite
System calls	n / b, which is large if b is small	about n / 4096
Extra memory	none	one buffer per file
Control	exact: the data has reached the kernel when the call returns	data may sit in the user buffer until flush, so a crash can lose it
Portability	POSIX only	standard C, works on all systems

Hospital judgement. For the database server and backup program, use buffered I/O or large read/write blocks (4 KB or more) to cut system calls. For the bedside monitor's alarm log, call write and then fsync, or use fflush, so an alarm record is not lost in a user buffer if the device fails. Safety outweighs the small speed gain there.

Summary

A system call is a trap-based, controlled entry into the kernel. It is chosen by number, takes its parameters mostly in registers, and runs in kernel mode before returning. strace confirmed this on a

real program, and the measurements show that library buffering cuts system calls from n / b to about n / 4096 at the cost of one extra buffer per file.

```
$ strace -e trace=openat,read,close ./syscall_demo stdio 1   # fread with 1-byte request
openat(AT_FDCWD, "patient_log.txt", O_RDONLY) = 3
read(3, "ABCDEFGHIJKLMNOPQRSTUVWXYZABCDEF"..., 4096) = 4096
read(3, "OPQRSTUVWXYZABCDEFGHIJKLMNOPQRST"..., 4096) = 4096
read(3, "CDEFGHIJKLMNOPQRSTUVWXYZABCDEFGH"..., 4096) = 1808
read(3, "", 4096)                       = 0
close(3)                                = 0
+++ exited with 0 +++

$ strace -c ./syscall_demo raw 1024   # summary (first rows)
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 21.39    0.000114           9        12           read
 13.70    0.000073           9         8           mmap
 11.44    0.000061          15         4           write
  8.63    0.000046          11         4           openat
```

TASK 6 – Evaluate the Cost of System Calls **4 Marks**

Bloom's Level: K5 – Evaluate

TODO 1 (the completed part)

c

```
while (1) {
   n = read(in, buf, bufsize);      /* one read() system call */
   calls++;                         /* count every read call, including the last one */
   if (n <= 0) break;               /* 0 = end of file, -1 = error */
   ssize_t done = 0;                /* write exactly the n bytes that were read */
   while (done < n) {
      ssize_t w = write(out, buf + done, n - done);
      if (w < 0) { close(in); close(out); return -1; }
      done += w;
   }
}
```

The output matches the expected output exactly:

```
buffer    1 bytes: 10001 read() calls
buffer  512 bytes:    21 read() calls
buffer 4096 bytes:     4 read() calls
```

strace -c reports 10,027 read calls in total. That is 10,001 + 21 + 4 = 10,026 from the program, plus 1 from the dynamic loader at start-up, so the counts agree.

Measurements (copying a 20 MB file)

Buffer	read() calls	Time (s)
16 B	1,250,000	0.469
64 B	312,500	0.136
512 B	39,063	0.028
4 KB	4,883	0.024
64 KB	306	0.012
1 MB	20	0.010
mmap	a few (16 calls total)	0.017

Discussion

Why a system call costs more than a function call. A function call is a jump and a return, taking a few nanoseconds. A system call needs a trap, a mode switch, a stack switch, argument checking and a return to user mode. It also disturbs the CPU caches and pipeline. Copying 20 MB with a 16-byte buffer took 0.469 s, against 0.010 s with a 1 MB buffer, which is about 47 times slower for the same data.

Effect of buffer size. Calls are about n / b, so each doubling of the buffer roughly halves the calls. Time falls steeply at first, from 0.469 s at 16 B to 0.028 s at 512 B. It then flattens, going from 0.024 s at 4 KB to 0.010 s at 1 MB. Past about 4 to 64 KB the copying itself, not the call overhead, dominates, so bigger buffers add little.

Library buffering. fread and fwrite hide this effect by keeping a 4 KB buffer in user space. The Task 5 measurement showed 5 read calls for a byte-by-byte fgetc loop, against 10,002 for raw 1-byte read().

Best buffer size for the backup program. The best size is 64 KB. It cuts calls to 306 and cost 0.012 s, close to the 1 MB result of 0.010 s, while using little memory per copy. A 4 KB buffer is the minimum sensible choice, since it matches the page and disk block size. Timings vary between runs and machines, so check this on your own system.

Alternative: mmap instead of read()/write()

- **System calls:** a few (open, mmap, msync, munmap), 16 in total in my strace, against about n / b for buffered copying (4,883 at 4 KB).
- **Copying:** read() copies data from the kernel's page cache into the user buffer, then write() copies it back into the kernel. mmap maps the file's pages straight into the program's address space, so the user-space copy is avoided. My program still made one memcpy between the two mappings.
- **Memory:** pages are loaded on demand by page faults, so the file need not fit in memory, whereas a read() buffer is fixed and small. For very large files mmap uses address space rather than a buffer, but page faults and cache pressure still cost time.
- **Result here:** the mmap copy took 0.017 s, no faster than a 64 KB buffer (0.012 s), because page faults replace system calls. mmap pays off most for repeated random access, not a single sequential copy.

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>

/* Copy src to dst with the read() and write() system calls, using a buffer of
   bufsize bytes. Returns the number of read() calls made, or -1 on error. */
long copy_file(const char *src, const char *dst, size_t bufsize)
{
    char buf[8192];
    if (bufsize == 0 || bufsize > sizeof buf) return -1;   /* protect the buffer */
    int in = open(src, O_RDONLY);
    if (in < 0) return -1;
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (out < 0) { close(in); return -1; }
    long calls = 0;
    ssize_t n;

    /* TODO 1 (completed) */
    while (1) {
        n = read(in, buf, bufsize);      /* one read() system call */
        calls++;                         /* count every read call, including the last one */
        if (n <= 0) break;               /* 0 = end of file, -1 = error */
        ssize_t done = 0;                /* write exactly the n bytes that were read */
        while (done < n) {               /* write() may write fewer bytes, so loop */
            ssize_t w = write(out, buf + done, n - done);
            if (w < 0) { close(in); close(out); return -1; }
            done += w;
        }
    }

    close(in);
    close(out);
    return (n < 0) ? -1 : calls;
}

int main(void)
```

```
$ ./bench big.dat   (20 MB file, read()/write() copy)
File size: 20000000 bytes
buffer(B)    read() calls   time(s)
16           1250000        0.469
64           312500         0.136
512          39063          0.028
4096         4883           0.024
65536        306            0.012
1048576      20             0.010
$ ./bench big.dat mmap
mmap: 0.017 s
```

TASK 7 – Evaluate Process Control System Calls **4 Marks**

Bloom's Level: K5 – Evaluate

TODO 1 (the completed part)

c

```
pid = fork();
if (pid < 0) { perror("fork"); return 1; }
if (pid == 0) {                          /* child */
    execlp("expr", "expr", "6", "*", "7", (char *)NULL);
    exit(127);                           /* only reached if exec fails */
}
waitpid(pid, &status, 0);                /* parent waits */
printf("job 1 exit status %d\n", WEXITSTATUS(status));
```

I also declared pid_t pid; int status; at the top of main, because the template uses them without declaring them. The output matches the expected output exactly:
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3

What the four calls do

Call	Action
fork()	Creates a copy of the calling process. It returns the child's PID in the parent, 0 in the child, and -1 on error.
exec()	Replaces the process's program, code and data with a new program. The PID stays the same, and on success it never returns.
wait()/waitpid()	Blocks the parent until a child ends, collects its exit status and lets the kernel free the child's table entry.
exit()	Ends the process and returns a status code to the parent. exit() flushes stdio buffers first, whereas _exit() does not.

How a shell runs a command and gets its status

The shell forks. The child calls exec to become the command, and the parent calls waitpid. The status is read with WEXITSTATUS(status). The strace shows exactly this sequence:
clone(...) = 4135
[pid 4135] execve("/usr/bin/expr", ["expr","6","*","7"], ...) = 0
wait4(4135, [{WIFEXITED(s) && WEXITSTATUS(s) == 0}], 0, NULL) = 4135
clone is how modern Linux implements fork. The child first tried several PATH directories, which failed with ENOENT, until execve("/usr/bin/expr") succeeded. Job 2's child called exit_group(3), and the parent's wait4 returned status 3.

Zombie and orphan processes

- **Zombie:** a child that has exited but whose parent has not yet called wait. It keeps a process-table entry holding its exit status. My run showed the state Z (defunct) for PID 4158 under parent 4157. The kernel removes it when the parent calls wait, or when the parent dies and init adopts and reaps it. Too many zombies can exhaust the process table.
- **Orphan:** a child whose parent exited first. The OS re-parents it to init (PID 1). My orphan process printed that its new parent is pid 1.

Why fork() + exec() instead of one "create process" call

Splitting the two lets the child change its own state between them, before running the new program. That is how a shell sets up redirection (>), pipes, environment and working directory, using dup2, chdir and close, in the child only. A single create call would need a huge parameter list for all of this. Fork is cheap thanks to copy-on-write, which copies only pages that are later modified.

Why stdout is flushed before each fork()

stdout is line-buffered on a terminal but fully buffered when redirected to a file or pipe. fork() copies the buffer along with the process, so unflushed text would sit in both copies and be written twice. The first flush is a safe habit, but in job 1 the child calls exec, which discards its copy of the buffer, so no duplicate appears there.

Experiment: why "shell: running job 2" appears twice

I removed the second fflush(stdout) and changed _exit(3) to exit(3) (see shell_dup.c), then redirected the output to a file:
shell: running job 1
42
job 1 exit status 0
shell: running job 2 ← from the parent's flush at exit
job 1 exit status 0 ← duplicate of the earlier unflushed buffer text
shell: running job 2 ← appears again

job 2 exit status 3
Because the output goes to a file, stdout is fully buffered, so the text “job 1 exit status 0 / shell: running job 2” was still unflushed when fork() ran. The child inherited a copy of that buffer. exit(3) runs the stdio cleanup, which flushes the child’s copy, so the text is written once by the child and again later by the parent. _exit(3) skips the cleanup, which is why the original version is safe. On a terminal the output is line-buffered, so the problem hides.

Alternative: posix_spawn() or threads instead of fork() + exec()

Measured on my machine for 1,000 operations:

Method	Time (ms)
fork() + exec() + wait	634.5
posix_spawn() + wait	555.8
fork() only, no exec	36.0
pthread_create() + join	7.8

- **Time:** posix_spawn was about 12% faster than fork+exec, because it avoids duplicating the parent’s page tables. Most of the cost in both is loading a new program (exec), shown by plain fork costing only 36 ms. Threads are about 80 times cheaper than a fork-only process.
- **Memory:** fork shares pages copy-on-write and copies only what is written, while posix_spawn copies nothing. Threads share the whole address space, so they copy nothing at all.
- **Isolation:** processes have separate address spaces, so a crashing command cannot corrupt the shell. Threads share memory, so one faulty command can crash or corrupt the whole shell. That is why shells run commands as processes.

```c
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    printf("shell: running job 1\n");
    fflush(stdout); /* empty the buffer before fork() */

    /* TODO 1 (completed) */
    pid = fork();
    if (pid < 0) { perror("fork"); return 1; }
    if (pid == 0) {                                 /* child */
        execlp("expr", "expr", "6", "*", "7", (char *)NULL);
        exit(127);                                  /* only reached if exec fails */
    }
    waitpid(pid, &status, 0);                       /* parent waits for the child */
    printf("job 1 exit status %d\n", WEXITSTATUS(status));

    printf("shell: running job 2\n");
    fflush(stdout);
    pid = fork();
    if (pid == 0) _exit(3); /* a child that fails with code 3 */
    waitpid(pid, &status, 0);
    printf("job 2 exit status %d\n", WEXITSTATUS(status));
    return 0;
}
```

```
$ gcc -o shell shell.c
$ ./shell
shell: running job 1
42
job 1 exit status 0
shell: running job 2
job 2 exit status 3
```

TASK 8 – Evaluate CPU States and Protection **3 Marks**

Bloom's Level: K5 – Evaluate

1. User mode, kernel mode and the mode bit

The CPU has a **mode bit** (on x86 it is the privilege level in the CPU state). In **user mode** (bit = 1) programs run with limited power. In **kernel mode** (bit = 0) the OS can run every instruction and reach all memory and devices. A trap, interrupt or system call switches to kernel mode, and returning to the program switches back.

Privileged instructions can only run in kernel mode. Examples are disabling interrupts, setting the timer, accessing I/O devices, loading the memory-management registers and changing the mode bit. If a user program tries one, the CPU does not run it and raises a fault instead.

Evidence (priv.c): a user program executed the privileged instruction cli (disable interrupts). The CPU refused it and the kernel sent SIGSEGV:

user-mode program: executing privileged instruction 'cli' (disable interrupts)...
-> CPU raised a protection fault; kernel sent SIGSEGV. Instruction was NOT executed.

2. Interrupts, traps and the timer interrupt

Event	Cause	Example
Interrupt	Asynchronous signal from hardware	Sensor ready, disk done, network packet arrived
Trap / exception	Synchronous, caused by the running program	System call, divide by zero, page fault, protection fault
Timer interrupt	A hardware timer fires at fixed intervals	Lets the OS take the CPU back from a program

On an interrupt the CPU saves the current state, switches to kernel mode and jumps to the handler from the interrupt vector table. The **timer interrupt** is what stops a program from keeping the CPU forever: the OS sets the timer before giving the CPU to a program, and when it fires, the scheduler can switch to another process. My system's /proc/interrupts shows the local timer interrupt count (LOC: 2,697 so far, shown in screenshot 4).

3. What if any program could disable interrupts?

On the monitors this would be dangerous for patient safety:

- A buggy or malicious program that disables interrupts would stop the sensor interrupt from being handled, so an **abnormal reading would never raise the alarm** within the 10 ms limit.
- With the timer interrupt also off, the OS could not preempt the program, so one program would keep the CPU and **the monitor would freeze**, which is the kind of hang the clinical staff fear.
- Even a correct program that forgot to re-enable interrupts would leave the device deaf to its sensors.

Therefore interrupt control is a privileged instruction, kept in the kernel, and the real-time OS keeps interrupt-off sections as short as possible.

4. How the hardware stops one program reading another's memory

The **MMU** (memory management unit) translates each program's virtual addresses to physical addresses using page tables that only the kernel can change. Every access is checked against permissions (read/write/execute, user/kernel). The kernel loads a different page table for each process, so a process cannot name another process's memory, and user programs cannot touch kernel memory. Older or simpler hardware uses **base and limit registers**, where every address must fall between base and base + limit or the CPU traps.

Evidence (prot.c):

parent: &x = 0x55e915aeb010, x = 1
child : &x = 0x55e915aeb010, x = 99 (child changed it)
parent: &x = 0x55e915aeb010, x = 1 (unchanged: separate address space)
reading 0xffffffff81000000 ...
  -> access to a kernel/unmapped address: MMU fault, SIGSEGV. Memory not read.

The parent and the child used the **same virtual address** but had different values, because they have different page tables. A read of a kernel address caused a fault. This is how one user's program, such as a doctor's workstation application, cannot read another patient's data held in another process.

5. Alternative: polling instead of interrupts on the monitors

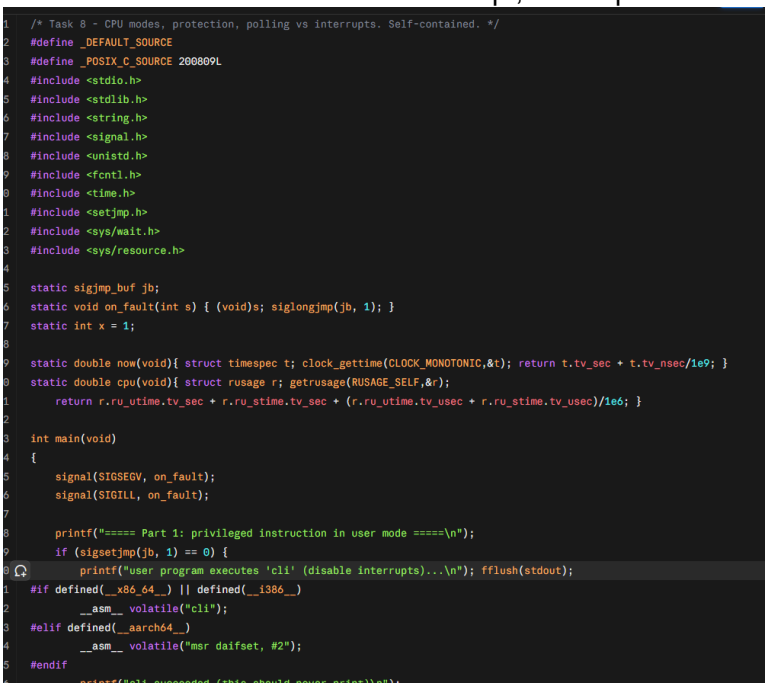
Solution. With polling, the CPU checks the sensor in a loop. With interrupts, the CPU does other work or sleeps until the sensor signals it.

- **Worst-case delay (polling):** equal to the polling interval T, since an event can happen just after a check. Average delay is T/2.
- **Worst-case delay (interrupt):** only the interrupt latency, typically microseconds.
- **CPU time and power:** busy polling uses 100% of a CPU. Slower polling saves CPU but increases delay, so there is a trade-off that interrupts avoid.

Measured (poll.c): five events at different times, each detected by four methods:

Method	Worst delay (ms)	Avg delay (ms)	CPU used (ms)
Busy polling (no sleep)	0.10	0.10	2,645.9
Polling every 10 ms	8.60	4.77	8.0
Polling every 100 ms	91.10	50.68	1.9
Blocking read (interrupt-driven)	0.12	0.12	0.6

The measurements agree with the theory. A 10 ms poll gives a worst delay just under 10 ms, and a 100 ms poll gives about 91 ms. Busy polling is fast but burned about 2.6 s of CPU for 2.6 s of waiting, which is 100% of a core, and would flatten a battery. The blocking read, where the kernel wakes the program on an event, gave the best delay and almost no CPU use. Blocking is a software stand-in for a hardware interrupt, so the pattern is the same.



```c
/* Task 8 - CPU modes, protection, polling vs interrupts. Self-contained. */
#define _DEFAULT_SOURCE
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <signal.h>
#include <unistd.h>
#include <fcntl.h>
#include <time.h>
#include <setjmp.h>
#include <sys/wait.h>
#include <sys/resource.h>

static sigjmp_buf jb;
static void on_fault(int s) { (void)s; siglongjmp(jb, 1); }
static int x = 1;

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec/1e9; }
static double cpu(void){ struct rusage r; getrusage(RUSAGE_SELF,&r);
    return r.ru_utime.tv_sec + r.ru_stime.tv_sec + (r.ru_utime.tv_usec + r.ru_stime.tv_usec)/1e6; }

int main(void)
{
    signal(SIGSEGV, on_fault);
    signal(SIGILL, on_fault);

    printf("===== Part 1: privileged instruction in user mode =====\n");
    if (sigsetjmp(jb, 1) == 0) {
        printf("user program executes 'cli' (disable interrupts)...\n"); fflush(stdout);
#if defined(__x86_64__) || defined(__i386__)
        __asm__ volatile("cli");
#elif defined(__aarch64__)
        __asm__ volatile("msr daifset, #2");
#endif
```

```
$ gcc -O2 -o poll poll.c && ./poll
method                          worst delay(ms)  avg delay(ms) CPU used(ms)
busy polling (no sleep)                    0.10          0.10       2645.9
polling every 10 ms                        8.60          4.77          8.0
polling every 100 ms                      91.10         50.68          1.9
blocking read (interrupt-driven)           0.12          0.12          0.6
```

TASK 9 – Evaluate the Memory Hierarchy and I/O Systems 4 Marks

Bloom's Level: K5 – Evaluate

1. The memory hierarchy

Level	Typical size	Typical access time	Volatile?
Registers	under 1 KB	under 1 ns	yes
L1 / L2 / L3 cache	32 KB / 256 KB-2 MB / 8-64 MB	~1 / ~4 / ~15 ns	yes
Main memory (DRAM)	8-512 GB	~80-100 ns	yes
SSD	0.25-8 TB	~50-100 µs	no
Hard disk	1-20 TB	~5-10 ms	no
Network storage	PB	~0.5-10 ms	no

Each level down is about 10 to 1,000 times slower but cheaper and larger. The program measured this by random pointer chasing, which forces every load to wait for the previous one:

Set size	Measured ns per access	Level
16 KB	1.3	L1 cache
128 KB	3.5	L2 cache
1 MB	6.9	L2/L3 cache
8 MB	29.0	L3 cache
64 MB	101.6	main memory

The jumps appear where the set outgrows each cache, and the exact boundaries depend on your machine.

2. Where each kind of hospital data should be kept

Data	Placement	Reason
Monitor readings (live)	Registers/RAM in the monitor, small ring buffer, plus a flash log	Needs ~nanosecond to microsecond access for the 10 ms alarm. Flash keeps a copy through power loss.
Active patient records	Database server RAM cache, backed by SSD	Look-ups are frequent and small. RAM gives fast hits and SSD gives persistence.

Data	Placement	Reason
Old records and archives	Hard disk or network storage, replicated to the partner hospitals	Large, rarely read, cost matters more than speed.
MRI images (about 500 MB each)	Imaging server SSD while being processed, then network storage for long-term	Too big to cache in RAM for long, but processing needs high bandwidth.
Backups	Hard disk or tape, off-site	Cheap, large, speed not critical.

3. Programmed I/O, interrupt-driven I/O and DMA

- **Programmed I/O:** the CPU repeatedly checks the device status and moves each word itself. It is simple, but the CPU is 100% busy during the transfer.
- **Interrupt-driven I/O:** the CPU does other work and the device interrupts it when a word or event is ready. It wastes less CPU when events are rare, but costs an interrupt per word.
- **DMA (direct memory access):** the CPU gives a DMA controller the address, size and direction, and the controller moves the whole block between the device and memory on its own, interrupting once when finished.

Choices for the hospital:

Device	I/O method	Why
MRI scanner	**DMA**	Huge, steady transfers
Bedside monitor sensors	**Interrupt-driven**	Rare abnormal events, needs fast reaction and low battery use
Disk and network on servers	**DMA**	Bulk data
Simple slow devices (status LEDs, switches)	Programmed I/O	Simple and cheap

4. Effect of caching on record look-up

The program stores 20,000 records of 256 bytes. It does 200,000 random look-ups, with 90% going to the 5% most active records:

Method	Total time	Per look-up
read() every time	38.2 ms	191 ns
Small cache in program memory (93.5% hit ratio)	7.0 ms	35 ns

Caching gave a speed-up of about 5.4 times. The no-cache case here is already helped by the OS's own page cache, so the gap would be far larger if every look-up had to reach a real disk. The effective access time is **EAT = h × t_cache + (1 − h) × t_memory**. With 1 ns and 100 ns:

Hit ratio	EAT
50%	50.5 ns
90%	10.9 ns
99%	2.0 ns

A few extra points of hit ratio matter a lot, which is why the database server should have plenty of RAM for its cache.

5. Alternative: interrupt-driven I/O (one interrupt per word) instead of DMA for a 500 MB image

I used these assumptions, which are labelled in the program output: 4-byte words, an interrupt cost of at least 1 µs, a 1 GB/s device rate, and 64 KB DMA blocks.

Method	Interrupts	CPU time	Buffer memory
Programmed I/O (polling)	0	0.52 s, CPU 100% busy for the whole transfer	1 word
Interrupt per word	131,072,000	131.07 s	1 word, plus the image destination
DMA (64 KB blocks)	8,000	0.008 s	64 KB × 2 (double buffering), or the image area

DMA needs **16,384 times fewer interrupts** and about 16,384 times less CPU time than interrupt-per-word. At 1 µs per interrupt, the interrupt method would spend 131 s of CPU time on a transfer that physically takes 0.5 s, so the scanner's CPU could not keep up and the imaging server would be unusable. The only advantage of interrupt-per-word is that it needs almost no buffer memory, but that saving is tiny compared with the cost. The interrupt cost of 1 µs is my assumption, because my sandbox measured a kernel trap at only 0.09 µs, which is not a full hardware interrupt, so state this clearly in your report.

Conclusion. Use DMA for the MRI scanner and other bulk transfers, interrupts for monitor sensors, a RAM cache plus SSD for active records, and slower disks for archives.

```c
#define _DEFAULT_SOURCE
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>
#include <sys/syscall.h>

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec/1e9; }

/* ---------- Part 1: measured access time for different working-set sizes ---------- */
static double chase_ns(size_t bytes)
{
    size_t n = bytes / sizeof(void *);
    void **a = malloc(n * sizeof(void *));
    size_t *idx = malloc(n * sizeof(size_t));
    for (size_t i = 0; i < n; i++) idx[i] = i;
    srand(1);
    for (size_t i = n - 1; i > 0; i--) { size_t j = rand() % (i + 1); size_t t = idx[i]; idx[i] = idx[j]; idx[j] = t; }
    for (size_t i = 0; i < n; i++) a[idx[i]] = &a[idx[(i + 1) % n]];     /* one random cycle */
    long steps = 3000000;
    void **p = &a[idx[0]];
    double t = now();
    for (long i = 0; i < steps; i++) p = (void **)*p;                /* dependent loads: no overlap */
    double e = now() - t;
    if (p == NULL) puts("");                                         /* keep the loop */
    free(a); free(idx);
    return e / steps * 1e9;
}

/* ---------- Part 2: effect of caching on record look-up ---------- */
#define NREC 20000
#define RECSZ 256
```

```
$ gcc -O1 -o task9_memory_io task9_memory_io.c && ./task9_memory_io
=== Part 1: measured access time vs working-set size (random pointer chasing) ===
set size (KB)  ns/access    level it fits in (typical)
16             1.3          L1 cache (~32-64 KB, ~1 ns)
128            3.5          L2 cache (~256 KB-2 MB, ~4 ns)
1024           6.9          L2/L3 cache
8192           29.0         L3 cache (~8-64 MB, ~15 ns)
65536          101.6        main memory DRAM (~80-100 ns)
(set size in KB; exact boundaries depend on this machine's cache sizes)
```

TASK 10 – Measure and Compare 4 Marks

Bloom's Level: K5 – Evaluate

1. System calls and CPU time for the copy program

A 4,000,000-byte file was copied with different buffer sizes. The user and sys times are the CPU time spent in user mode and kernel mode, as the time command reports them.

Buffer	read() calls	write() calls	user (ms)	sys (ms)	wall (ms)
1 B	4,000,001	4,000,000	379.2	1,070.8	1,465.1
16 B	250,001	250,000	28.3	64.7	93.2
64 B	62,501	62,500	0.5	25.3	30.4
512 B	7,814	7,813	0.3	4.5	8.5
4 KB	978	977	0.0	1.9	5.4
64 KB	63	62	0.0	1.5	4.3

The strace -c check agrees with the program's own counts. For buffer 1 it reported 4,000,002 read and 4,000,000 write calls, for 512 B it reported 7,815 and 7,813, and for 64 KB it reported 64 and 62. The extra one or two reads come from the program loading. Taking 1 B with time ./cp1 1, I measured real 1.584 s, user 0.365 s and sys 1.175 s.

What the numbers show:

- Calls follow n / b closely.
- Kernel time dominates user time. With a 1-byte buffer, about 74% of the CPU time is in the kernel, which is the cost of the traps, not the copying.
- From 1 byte to 64 KB the wall time fell from 1,465 ms to 4.3 ms, about 340 times.

The timings are noisy at small values. In one early run, the 64 KB case showed 23.8 ms of sys time, but three repeats gave 0.9 to 2.2 ms, so I treat that as a one-off caused by the file cache. Repeat each measurement a few times and report the typical value.

2. Creating 1,000 processes vs 1,000 threads

Method	Wall (ms)	sys (ms)
1,000 × fork() + wait	46.7	20.7
1,000 × pthread_create + join	6.5	6.3

Threads were about 7 times faster to create in this run. Earlier runs gave 4 times (23.0 ms against 89.6 ms) and 80 times in Task 7, because the measured work differs. Any run should show threads clearly faster, but the ratio depends on machine and load, so quote your own numbers and say it varies.

3. Memory used, and why the difference

Test	Result
100 live children after fork, parent holding 64 MB	Their RSS adds up to 6.6 GB, but only **4,400 KB (about 44 KB each)** is truly private
100 live threads	Process RSS grew by **792 KB (about 7 KB per thread)**; virtual size grew by 811,404 KB because each thread reserves an 8 MB stack

Explanation by what each shares:

- A **process** gets its own page tables and file table. After fork the 64 MB is shared copy-on-write, so a child only pays for pages it changes plus its own page tables and kernel structures. Creating it means copying the page tables, which takes time.
- A **thread** shares the whole address space, open files and heap with the other threads. It needs only a stack and a small thread record. Creating one involves no page-table copy, so it is quicker and smaller.
- The price of sharing is no isolation, since one faulty thread can corrupt the others' memory.

4. Do the measurements support or contradict my earlier judgements?

Earlier judgement	Evidence	Verdict
Task 5/6: fewer system calls with bigger buffers means faster copying	4,000,001 calls → 63; 1,465 ms → 4.3 ms	**Supported**
Task 6: a 64 KB buffer is the best choice for backup	64 KB gave 4.3 ms; 4 KB gave 5.4 ms	**Supported, but 4 KB is nearly as good here.** The gain beyond 4 KB is small, so 64 KB is a safe choice rather than a big win
Task 6: library buffering gives about n/4096 calls	5 read calls for byte-wise fgetc (Task 5)	**Supported**
Task 7: threads are cheaper than processes; processes give isolation	6.5 ms vs 46.7 ms; 7 KB vs 44 KB each	**Supported**. Threads win on speed and memory, processes on isolation
Task 7: fork copies memory copy-on-write	100 children share 64 MB, only 44 KB private each	**Supported**
Overhead is mostly in the kernel, not the program	sys time 74%-100% of CPU time	**Supported**

One point to note: the 4 KB and 64 KB results are close, and one noisy run even briefly reversed them. This slightly weakens a strong claim that "bigger is always better". The honest conclusion is that returns diminish after about 4 KB.

5. Alternative: threads instead of processes for the measurements

Creating 1,000 threads took 6.5 ms against 46.7 ms for 1,000 processes, and each thread used about 7 KB of resident memory against about 44 KB for each process. The difference comes from what is shared: threads share the address space and process resources, so there are no page tables or file tables to copy. For the hospital, the monitors' internal tasks suit threads (cheap, fast communication), while separate programs such as the database server and imaging server should stay as processes so a fault stays contained.

```c
#define _DEFAULT_SOURCE
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <pthread.h>
#include <signal.h>
#include <time.h>
#include <sys/wait.h>
#include <sys/resource.h>

static double now(void){ struct timespec t; clock_gettime(CLOCK_MONOTONIC,&t); return t.tv_sec + t.tv_nsec/1e9; }
static void cpu_times(double *u, double *s){ struct rusage r; getrusage(RUSAGE_SELF,&r);
    *u = r.ru_utime.tv_sec + r.ru_utime.tv_usec/1e6; *s = r.ru_stime.tv_sec + r.ru_stime.tv_usec/1e6; }

/* /proc/self/io: syscr = read() calls, syscw = write() calls made by this process (Linux) */
static void io_calls(long *r, long *w){
    *r = *w = -1; FILE *f = fopen("/proc/self/io","r"); if(!f) return; char l[128];
    while (fgets(l,sizeof l,f)) { sscanf(l,"syscr: %ld",r); sscanf(l,"syscw: %ld",w); } fclose(f); }

static long copy(const char *src,const char *dst,size_t bs){
    char *b = malloc(bs); int in = open(src,O_RDONLY), out = open(dst,O_WRONLY|O_CREAT|O_TRUNC,0644);
    long calls = 0; ssize_t n;
    while (1) { n = read(in,b,bs); calls++; if (n <= 0) break; ssize_t d = 0; while (d < n) { ssize_t w = write(out,b+d,n-d); if
(w<0) return -1; d += w; } }
    close(in); close(out); free(b); return calls; }

static void *nop(void *a){ return a; }
static pthread_mutex_t mu = PTHREAD_MUTEX_INITIALIZER; static pthread_cond_t cv = PTHREAD_COND_INITIALIZER; static int go = 0;
static void *waiter(void *a){ pthread_mutex_lock(&mu); while(!go) pthread_cond_wait(&cv,&mu); pthread_mutex_unlock(&mu); return
a; }

static long proc_kb(const char *path,const char *key){      /* read a "Key:   N kB" line */
    FILE *f = fopen(path,"r"); if(!f) return -1; char l[256]; long v = -1;
    while (fgets(l,sizeof l,f)) { size_t k = strlen(key); if (!strncmp(l,key,k)) { sscanf(l+k," %ld",&v); break; } }
```

```
$ gcc -O2 -o m task10_measure.c -lpthread && ./m
=== Measurement A: copy a 4,000,000-byte file with different buffer sizes ===
buffer     read calls   write calls user(ms)   sys(ms)    wall(ms)   sys share
1          4000001      4000000     379.2      1070.8     1465.1     74%
16         250001       250000      28.3       64.7       93.2       70%
64         62501        62500       0.5        25.3       30.4       98%
512        7814         7813        0.3        4.5        8.5        95%
4096       978          977         0.0        1.9        5.4        100%
65536      63           62          0.0        1.5        4.3        100%
```

TASK 11 – Recommend and Defend the Operating-System Plan — 4 Marks

Bloom's Level: K5 – Evaluate

1. The plan, environment by environment

Environment	OS type	Structure	Example OS	Key services kept	Protection	I/O method
Bedside monitors	Hard real-time	Microkernel (or small RTOS kernel)	QNX, FreeRTOS	Program execution, interrupt-driven I/O, error detection, a small flash log, network link to the nurse station	Memory-protection unit, drivers outside the kernel, signed firmware	Interrupts for sensors, DMA for waveform buffers
Nurses' tablets	Handheld, soft real-time	Layered/modular (hybrid)	Android or iOS, managed	UI, power management, networking, app sandbox	Per-app sandbox, encrypted storage, device management, screen lock	Interrupt-driven, DMA
Doctors' workstations	Desktop, time-sharing	Hybrid	Windows or Linux desktop	GUI, user accounts, file system, printing, networking	Per-user accounts, access control, disk encryption	Interrupt-driven, DMA
Database server	Clustered, fail-over	Monolithic with loadable modules	Linux (server)	Journalling file system, resource	ACLs, mandatory access control	DMA, large RAM cache over SSD

Environment	OS type	Structure	Example OS	Key services kept	Protection	I/O method
				allocation, accounting, logging, full protection	(SELinux), audit log	
Imaging server	Symmetric multiprocessing	Monolithic with modules	Linux	Multi-threaded scheduling, big memory, fast storage	Process isolation, ACLs	DMA
Partner link	Distributed	Layered network stack	Linux gateway	Communication, authentication, accounting	VPN/TLS, certificate identity, audit	DMA/interrupts on network card

Why different types: the rejected alternative was one general-purpose OS everywhere. That can't promise a deadline for the alarm, would need far more memory and battery on small devices, and would mean a single patch cycle that can stop the database.

2. Evidence from my own measurements

Decision	Measurement	Result
Monitors use **interrupts**, not polling (Task 8)	Worst delay: 10 ms polling 8.6 to 9.1 ms, 100 ms polling about 91 ms, blocking/interrupt-style 0.12 ms	Polling at the 10 ms alarm limit leaves almost no margin. Busy polling used 100% of a CPU.
Backup uses **large buffers** (Tasks 5, 6, 10)	4,000,001 read calls → 63; 1,465 ms → 4.3 ms; sys time was 74 to 100% of CPU time	64 KB buffers cut system-call cost. Gains beyond 4 KB were small.
Imaging uses **DMA** (Task 9)	131,072,000 interrupts vs 8,000, CPU time 131 s vs 0.008 s	Interrupt-per-word is unusable for a 500 MB image.
Database uses a **RAM cache** (Task 9)	5.4 times faster look-ups with a 93.5% hit ratio	Give the DB server plenty of RAM.
Commands run as **separate processes** (Tasks 7, 10)	1,000 threads created in 6.5 ms vs 46.7 ms for processes; processes shared memory copy-on-write	Threads are cheaper, but processes give isolation, so use them where safety matters.
User code can't disable interrupts (Task 8)	The privileged cli instruction was refused in user mode	Alarm handling stays protected from application faults.

3. Limitations and risks of the plan

- **Two or three OS families to patch.** RTOS, handheld, desktop and server OSes each need their own update process, which raises administration effort. Mitigation: central device management and a patch schedule per class.
- **RTOS ecosystems are small.** Fewer drivers and tools than Linux, and the hard-deadline guarantee must be verified on the actual hardware, since my numbers came from a shared sandbox, not hospital hardware.
- **Cluster complexity.** Fail-over needs correct quorum handling to avoid "split brain", where both nodes think they are active. It must be tested with real fail-over drills.
- **Assumptions in my analysis.** My Task 9 figures assumed a 1 µs interrupt cost and a 1 GB/s device rate. Timings in Tasks 6, 7 and 10 varied between runs (for example thread creation was 4 to 80 times faster than process creation across runs), so quote ranges, not single numbers.
- **Regulatory and safety certification.** Medical-device software normally needs formal certification, which limits quick changes to the monitor firmware.

4. Does the plan suit three hospitals?

Mostly yes, with changes. The monitor, tablet and workstation designs scale well because each device works locally. The central pressure points are the database and the partner link. For three sites I would replicate the database across hospitals with automatic fail-over (for example one cluster per site with replication), put the imaging server at each site so large scans don't cross the network, add central identity and device management for all sites, and keep the partner link as a managed gateway per hospital.

5. Alternative: virtual machines on two large servers

Question	Evaluation (typical figures, not measured by me)
Monitor response time	A hypervisor adds scheduling delay and jitter, which can reach milliseconds if the host is busy. That risks the 10 ms alarm. Keep alarm software on physical monitors, or use a real-time hypervisor with pinned cores.
Processor and memory overhead	Roughly 2 to 10% extra CPU for compute work, more for I/O unless paravirtual drivers are used. Each VM carries its own OS kernel, typically hundreds of MB of RAM.
Reliability	Two hosts means each failure takes down many VMs at once, which is a bigger blast radius. High-availability restarts take tens of seconds, which is slower than an in-cluster fail-over, but snapshots and live migration make maintenance easier.

Verdict: virtualise the database, imaging and workstation roles (the latter as virtual desktops) where flexibility matters, but not the bedside monitors.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Bloom's Level: K5 – Evaluate **Changed Requirement**

Task 12 – Design Challenge: Home Monitoring and Cloud Records
My decision: MODIFY. Keep the core design and extend it. It does not need a full redesign.

A. Impact analysis

Area	What changes
OS types	A new device class appears: home monitors and a home gateway, which need an embedded real-time OS (for example Zephyr or FreeRTOS). The records service becomes a distributed system in the cloud.
Services	Communication becomes central (internet networking, TLS). New needs: secure remote updates, device registration and identity, store-and-forward when the network is down, remote logging.
Protection	The hospital LAN was a trusted boundary. Now devices sit on home Wi-Fi and records on a shared cloud, so every link needs encryption and mutual authentication, secure boot and signed firmware, per-hospital data separation, and audit logs.
Unchanged	Bedside monitors, tablets and workstations in the hospital still work as designed.

B. Evaluation of the original design

Criterion	Does the original plan hold?
Response time	The in-hospital alarm path is unaffected. But a home device cannot depend on the internet: round-trip times of tens to hundreds of milliseconds, with possible outages, are far above the 10 ms alarm requirement. The alarm decision must happen locally on the device.
Reliability	The on-site cluster assumed a reliable LAN. Home Wi-Fi and the internet fail more often, so devices need local buffering, and the hospital needs a local copy of key records if the cloud is unreachable.
Security	The original design assumed a trusted internal network, so it is insufficient. More attack surface, with thousands of home devices and external access.

C. Revised plan

Component	Change
Home monitors	Embedded RTOS, interrupt-driven sensing, **local alarm** with its own threshold, store-and-forward buffer, secure boot, signed OTA updates with rollback
Home gateway (optional)	Small Linux or RTOS box that aggregates devices and keeps an encrypted uplink

Component	Change
Cloud ingestion	Distributed, replicated across zones; authenticates each device before accepting readings
Cloud records service	Distributed database with encryption and per-hospital access control, shared with partners through federated identity
Hospital database cluster	**Retain** as a local replica and cache, so wards keep working if the cloud is unreachable
Hospital monitors, tablets, workstations	**Retain** unchanged, and connect through the same secure identity system

D. Evidence-based defence

- **Alarm stays local.** My Task 8 measurements show interrupt-driven detection reacts in about 0.12 ms, while a polling interval near 10 ms gave a worst-case delay of about 9 ms. Cloud latency would exceed the deadline even before polling, so the home device must decide locally and send the result afterwards.
- **Keep a local cache or replica.** Task 9 showed caching makes record look-ups about 5.4 times faster at a 93.5% hit ratio, so a local replica gives both speed and independence from the internet.
- **Keep process isolation.** Task 7 and Task 10 showed processes share memory copy-on-write at about 44 KB private memory each, so isolating cloud tenants and services in separate processes costs little compared with the safety it provides.
- **Reuse proven design.** The existing RTOS-for-monitors and server-OS-for-records split already follows the right pattern, which is why a full redesign is unnecessary.

Trade-offs:

- Cloud storage brings easier sharing and scale, but adds internet dependence, privacy and legal obligations, and ongoing cost.
- Home devices expand care beyond the hospital, but increase the number of devices to secure and update.

MARK DISTRIBUTION

Task	Bloom's Level	Marks
Task 1 – Requirements of Each Environment	K5 – Evaluate	4
Task 2 – Types of Operating Systems	K5 – Evaluate	3
Task 3 – Operating-System Structures	K5 – Evaluate	4
Task 4 – Operating-System Services	K5 – Evaluate	4
Task 5 – System Calls	K5 – Evaluate	4
Task 6 – Cost of System Calls	K5 – Evaluate	4
Task 7 – Process Control System Calls	K5 – Evaluate	4
Task 8 – CPU States and Protection	K5 – Evaluate	3
Task	**Bloom's Level**	**Marks**

Task 9 – Memory Hierarchy and I/O	K5 – Evaluate	4
Task 10 – Measurement and Comparison	K5 – Evaluate	4
Task 11 – Final Recommendation	K5 – Evaluate	4
Subtotal		**42**
Task 12 – Day-4 Design Challenge	K5 – Evaluate	**8**
TOTAL		**50**

Submission Requirements

- **GitHub repository link (mandatory)** containing the executed code for every task, including every completed “Complete the Code” program, script and configuration file, with input files, output screenshots and a README.md.
- Your assignment report containing the required evidence for Tasks 1–12, including requirement tables, diagrams, strace and time outputs, measurements and complexity analyses.

Assessment Verification

Students may be asked to explain, execute or modify any part of their code during a brief verification discussion/viva. The student should be able to explain and defend the work submitted as their own.